

“GPU 공동 사용 플랫폼” 구현 방식 검토 결과보고서

컴퓨터공학전공 2020112028 정상원

1. 검토 개요

1.1 문제 정의

교내 각 연구실은 GPU 를 각자 구매해 보유하고 있다. 장비는 구매한 연구실의 것이고, 그 연구실이 쓰지 않는 시간에도 다른 구성원은 접근할 수 없다. 그 결과 한쪽에서 값비싼 연산 자원이 밤과 주말 내내 노는 동안, 다른 쪽에서는 장비가 없는 연구실이 실험을 미루거나 외부 클라우드에 비용을 지출한다. 학교 전체가 보유한 연산 능력의 총합은 적지 않은데, 그 자원이 소유 단위로 잘게 갈라져 있어 정작 필요한 곳에 닿지 못한다.

‘GPU 공동 사용 플랫폼’은 이 단절을 메우려는 시도로서 기획되었다. 각 연구실이 가진 장비를 하나의 체계로 묶어 소유 연구실이 쓰지 않는 시간에 다른 구성원이 쓸 수 있게 하는 시스템이며, 목표는 교내에 흩어진 GPU 자원의 효율적 분배다. 다만 이 목표 하나를 이루는 데에도 서로 다른 결정이 여럿 필요하다. 사용자에게 무엇을 어떤 형태로 내놓 것인가, 요청이 겹칠 때 누구에게 먼저 줄 것인가, 장비를 내놓은 연구실은 그 대가로 무엇을 받는가. 본 보고서는 이 가운데 첫 번째인 구현 방식을 다룬다. 나머지 두 질문은 구현 방식이 정해진 뒤에야 구체적인 형태를 갖기 때문이다.

이 검토에는 조건이 붙는다. 개발에 투입할 수 있는 인력과 기간이 한정되어 있어, 여름 개발 TF 단위의 자원을 넘어서는 계획은 완성에 이르지 못할 가능성이 크다. 그래서 이 검토는 무엇을 만들 것인가보다 무엇을 만들지 않을 것인가를 먼저 보며, 이미 성숙한 오픈소스가 존재하는 영역은 직접 개발하지 않고 채택하는 것을 원칙으로 삼는다. 4 장에서 스케줄러를 직접 만들지 않는 근거가 여기에 있고, 학교 안에 이미 쌓인 것을 먼저 살피는 이유도 같다. 이전에 수행된 개별연구 「플랫폼 엔지니어링을 적용한 연구용 Kubernetes 기반 IDP 구축」의 성과를 활용할 수 있다면 연구와 교육 측면의 연속성이 확보되고 무엇보다 개발 비용이 줄어든다. 어느 부분이 실제로 재사용 가능한지는 구현 방식에 따라 달라지므로, 이 문제는 5 장에서 방식별로 나누어 다룬다.

이상을 종합하면 본 검토가 답해야 할 공학적 문제는 다음과 같다. 한정된 개발 인력과 기간 안에서, 사양이 제각각인 교내 GPU 장비를 여러 연구실이 공동으로 사용하게 하되, 연구자가 기존 작업 방식을 크게 바꾸지 않아도 되고, 자원 배정의 근거를 설명할 수 있는 시스템을 설계하는 것이다. 여기에는 개발 비용, 이질적 장비의 수용, 낮은 사용자 부담, 설명 가능한 배정이라는 네 가지 요건이 들어 있다. 이후 모든 구현 방식은 이 네 요건에 비추어 평가한다.

1.2 선행 사례

같은 문제를 먼저 마주한 대학들이 있는데, 국내에서 규모가 가장 잘 알려진 곳이 성균관대학교 슈퍼컴퓨팅센터다. 2022 년 9 월 1 일 기준 자료에 따르면 NVIDIA A100 SXM4 80GB 40 장을 Dell PowerEdge XE8545 서버 10 대에 나누어 신품 스토리지 1,910TB 를 함께 운용한다. 사용자는 신청을 통해 계정을 발급받고 웹사이트에서 연산 세션을 실행하며, 요금은 GPU 당 시간당 2,200 원으로 세션 종료 후 전자세금계산서가 발행된다. CPU 자원만 쓰는 경우는 무상이고 교내 사용자는 연구비로 사용요금을 지출할 수 있다. 무엇보다 이 센터는 Lablup 의 Backend.AI 를 도입해 쓰고 있으며 시스템을 직접 개발하지 않았다. (별첨 3)

다른 국내 대학도 비슷한 경로를 밟았다. 국민대학교는 Backend.AI 를 도입했고, 조윤호 교수의 기록에 따르면 GPU 서버 3 대로 시작해 80 명 이상의 학생이 동시에 모델링 실습을 수행할 수 있게 되었으며 전담 관리 인력 없이 운영되고 있다. KAIST AI 대학원은 VESSLA I 를 도입해 GPU 800 장 이상을 20 개 연구팀과 대학원생 300 명 이상이 사용하게 했고, 월간 GPU 사용 시간과 동시 작업 수에 할당량을 두었다. 이 사례에는 도입 이전 상태가 함께 기록되어 있는데, 우리 상황과 겹치는 부분이 많다. Kubernetes 의 복잡도 때문에 접근 자체가 어려웠고, 접근에 성공한 사용자는 GPU 를 몇 주씩 점유했으며, 할당량 정책과 사용량 관측 수단이 없어 그런 상태가 드러나지 않았다. 그 밖에 고려대학교는 공동기기원이 GPU 클라우드 서비스를 출범하며 10 년간 300 억 원 투입 계획을 밝혔고, 서울대학교 공과대학은 GPU 클러스터 운영에 MIG 를 도입했으며, POSTECH 은 A100 80GB 를 포함한 약 700 장 규모의 클러스터를 25 개 연구실에 전면 무상으로 개방하고 있다.

국내 사례가 대체로 완성된 플랫폼 제품을 들여오는 형태였다면 해외 대학은 다른 경로를 택했다. 미국 주요 대학의 연구 전산 환경은 Slurm 기반 운영이 표준이며 Princeton, Northwestern, Utah 등이 여기에 해당한다. 그중 유타대학교 CHPC 와 메릴랜드대학교 UMIACS 는 우리가 마주한 문제를 정면으로 다룬 운영 방식을 설정 문서와 함께 공개하고 있다. 연구실이 장비를 공동 클러스터에 넣되 소유권은 유지하는 구조로, 소유 연구실이 쓰지 않는 동안에는 다른 사용자가 그 장비를 쓰고 소유 연구실이 작업을 제출하면 우선권을 되찾는다. 연구실이 왜 장비를 내놓아야 하는가라는 질문에는 이미 확립된 답이 있다.

경로는 갈렸지만 공통점이 하나 있다. 조사한 어느 곳도 시스템 자체를 직접 개발하지 않고, 상용 제품을 구매했거나 기존 오픈소스 위에 자기 기관의 규칙을 얹었다. 각 기관이 실제로 만든 것은 요금 체계와 할당량 정책과 소유권 규칙이었다. 이는 1.1 에서 세운 원칙과 같은 방향을 가리키며, 우리가 새로 만들어야 하는 것도 시스템보다는 우리 학교 사정에 맞는 정책과 운영 방식을 반영한 패치에 가까울 가능성이 크다.

1.3 구현 방식의 분류

그렇다면 어떤 시스템 위에 그 정책을 얹을 것인지부터 정해야 한다. 이에 대한 선택지는 플랫폼이 어느 계층에서 개입하고 그 결과 사용자가 무엇을 받는지를 기준으로 네 가지로 나뉘며, 개입 계층이 낮아질수록 사용자가 받는 단위는 커진다.

가장 깊이 개입하는 쪽부터 보면, 광역 GPU 가상화는 GPU 연산 자체를 네트워크 너머의 서비스로 제공해 사용자가 장비를 의식하지 않게 하는 방식이나, 학습 워크로드를 광역망 너머로 보내 실행하는 사례는 아직 찾지 못했다. 국내 GPU 가상화는 같은 구조를 교내망으로 한정해 프로그램은 사용자 자리에서 실행하되 GPU 호출만 다른 장비로 전달하는 방식으로, rCUDA 를 비롯한 연구 프로젝트가 이를 구현해 왔다. 작업 스케줄링은 개입 지점을 한 단계 올려 작업을 제출받아 순서를 정하고 GPU 장비에서 실행한 뒤 결과를 돌려주는 방식이며, Slurm 을 쓰는 대다수 대학 HPC 센터와 앞서 언급한 미국 대학들이 여기에 속한다. 마지막으로 가상 인스턴스 임대는 GPU 가 포함된 실행환경을 통째로 배정하고 그 안에서 벌어지는 일에 관여하지 않는 방식으로, 성균관대학교와 국민대학교, KAIST AI 대학원이 이렇게 운영하며 AWS 와 GCP 등 상용 클라우드도 같은 형태다.

네 방식은 기술적 난이도, 사용자 부담, 개발 비용에서 서로 다른 대가를 요구한다. 이하에서는 1.1 에서 정의한 네 요건에 비추어 각 방식을 차례로 검토한다.

2. 각 방식에 대한 검토

2.1 광역 및 구내 GPU 가상화

두 방식은 병목의 성격이 같고 정도만 다르므로 함께 다룬다. 프로그램은 사용자 쪽에 두고 GPU 호출만 다른 장비로 전달하는 구조여서, 사용자 코드를 손대지 않아도 되고 사용자는 GPU가 어디에 있는지 알 필요조차 없다. 이 방식은 네 방식 가운데 추상화 수준이 가장 높고 이론상 사용자 부담도 가장 작다.

그러나 전달 과정에서 이에 대한 대가를 치르게 된다. 연산 하나하나가 그대로 네트워크를 건너기 때문에, 메모리 할당과 커널 실행과 메모리 복사가 전부 개별 왕복이 되고 그 횟수는 초당 수천 회에 이른다. rCUDA 계열의 측정에 따르면 왕복 비용에 의해 네이티브 대비 1.42 배에서 5.54 배 느려진다. 이마저도 InfiniBand 같은 RDMA 계열 인터커넥트가 필요하고, 광역망에서는 왕복 지연이 커널 실행 시간보다 수만 배 크기 때문에 애초에 측정이 성립하지 않는다.

교내 전 연구실에 RDMA 인터커넥트를 부설하는 일은 1.1 에서 정한 제약을 한참 벗어난다. 이 계열의 연구가 2010년대 중반 이후 크게 줄었다는 점도 함께 볼 만한데, 인터커넥트를 이미 갖춘 환경에서조차 이 방식이 표준으로 자리잡지 못했다는 신호로 읽을 수 있다. 이하 장에서는 더 이상 이러한 두 가지 방식을 다루지 않는다.

2.2 작업 스케줄링

작업 스케줄링 방식에서 플랫폼은 사용자로부터 실행할 내용과 함께 GPU 몇 장을 몇 시간 동안 쓸 것인지를 전달받아, 순서를 정한 뒤 GPU 장비에서 실행하고 그 결과를 사용자에게 돌려준다. 이 구조에서는 자원이 놀지 않는다. 작업 단위로 배정하므로 하나가 끝나면 즉시 다음이 들어가고, 활용률로만 보면 네 방식 중 가장 높다. 대기열이 겹으로 드러나므로 누가 왜 기다리고 있는지 설명할 수 있고, 그 위에서 우선순위와 형평성과 예산을 표현하는 수단도 오랜 기간에 걸쳐 성숙해 있다. 반대로 사용자는 자기 작업을 제출 가능한 형태로 정리해야 한다. 실행 중인 작업에 접근하기 어려워져 사람이 붙어 들여다보는 작업 방식과는 잘 맞지 않는다. 그리고 자원을 회수하는 기능을 쓰기 시작하면 실행 중이던 다른 작업이 중단되므로, 누구의 작업을 언제 멈출 것인지에 대한 행정적 기준이 필요해진다.

2.3 가상 인스턴스 임대

가상 인스턴스 임대 방식에서 플랫폼은 사양을 고른 사용자에게 실행환경 하나를 통째로 배정하고, 그 안에서 벌어지는 일에는 관여하지 않는다. 사용자는 실제 장비에 준하는 환경을 받아 그 안에서 원하는 대로 작업한다. 이 방식에서는 사용자가 아무것도 바꾸지 않고 쓰던 도구를 그대로 쓰며 익숙한 순서로 일한다. 사양이 사전에 고정되어 있어 다른 사용자의 작업이 내 작업을 방해하는 경우도 적다. 상용 클라우드에서 이미 익숙한 모델이라 사용자가 이해하는 데 드는 비용도 거의 없다. 대신 자원을 스케줄링만큼 촘촘히 쓰지는 못한다. 인스턴스를 켜두고 아무것도 하지 않아도 자원이 묶여 있고, 사양을 바꾸려면 재시작이 필요한 경우가 많다. 배정 단위가 크면 자원을 잘게 나누어 쓰기도 어렵다.

3. 선행 연구 조사

3.1 조사 이전의 전제

그런데 검토를 시작한 시점에 알지 못했던 사실이 세 가지 있었고, 그것들이 결론을 좌우한다. 이전 개별연구를 수행한 학생들도, 최근까지의 본인도 이를 전제하지 않은 상태에서 설계를 진행했다. 이는 이전 개별연구 보고서의 참고문헌이 PerfSeer와 Kubernetes Dynamic Resource Allocation 두 건뿐이고 기존 자원 관리 시스템에 대한 조사가 들어 있지 않다는 점에서도 확인된다. 방식 비교에 앞서 세 가지를 순서대로 제시한다.

3.2 Slurm

첫 번째는 Slurm 의 존재다. SchedMD 가 GPLv2 로 배포하는 배치 스케줄러로 20 년 이상의 운영 이력을 가지며, 세계 상위 500 대 슈퍼컴퓨터의 절반 이상이 이를 사용하고 대학 HPC 환경에서는 사실상 기본값에 해당한다.

이전 검토에서는 API 방식을 구현하려면 TensorFlow 나 PyTorch 같은 애플리케이션 계층에서 호출을 가로채 별도의 API 를 만들어야 한다고 판단하고 있었다. 그런데 그 전체를 감싸는 계층에, 개별연구의 코드보다 훨씬 성숙하고 즉시 사용 가능한 구현이 이미 자리잡고 있었다. 장비는 GRES 타입 선언으로 다루고, 연구실별 지분은 계정 계층과 fairshare 로 표현하며, GPU-분 단위의 예산 상한과 소유 연구실 우선권도 전부 설정값으로 커스텀이 가능하다. 우리가 설계하려던 정책 표현 수단이 통째로 갖추어져 있다.

다만 Slurm 이 있다고 해서 이전 개별연구에서 조사된 정책을 그 위에 얹는 일까지 자동으로 되는 것은 아니다. 기존 PerfSeer 기반 예측은 단일 GPU 에서 측정한 데이터로 학습했고 PerfSeer 데이터셋 자체가 RTX 3090 단일 장비 기준이다. 지도교수가 확인하고자 한 사안, 즉 '두 장 이상에서도 동작하는지, 성능이 어느 정도 나오는지'에 관한 내용은 장비를 늘려 돌려보는 것만으로 답이 나오지 않는다. 다중 GPU 환경에서는 병렬화 전략, 집합통신 비용, 인터커넥트 종류, 확장 효율이 결과를 지배하는데, 단일 GPU 계산 그래프에는 이 가운데 어느 것도 표현되어 있지 않기 때문이다. 입력 특징 자체를 새로 설계하고 데이터셋도 새로 수집해야 하므로 이는 새로운 연구에 해당한다.

이 영역에는 선행 연구가 이미 여럿 있고, 오히려 최근 몇 년 사이 활발히 학술활동이 이어진 분야이다. ScaleDL 은 분산 딥러닝 워크로드의 실행 시간 예측을 다루고, HETHUB 은 이기종 가속기 환경을 위한 분산 성능 예측기와 자동 병렬 계획기를 제시하면서 AMD 와 NVIDIA 가 혼합된 768 장 규모 클러스터에서 대형 모델을 학습한 사례를 보고한다. Cephalo 와 HARP 도 이기종 클러스터에서의 학습 최적화를 다룬다.

그럼에도 우리 환경에 필요한 연구는 찾지 못했는데, 이는 두 가지로 나누어 설명할 수 있다. 하나는 기존 연구를 우리 상황에 그대로 옮겨오기가 간단하지 않다는 점이다. 위 연구들은 데이터센터급 가속기와 고대역 인터커넥트를 전제하지만, 기증받은 소비자용 GPU 가 일반 이더넷으로 묶인 환경은 제약조건이 전혀 다르다. 다른 하나는 우리가 예측해야 하는 대상이 기존 연구와 다르다는 점이며, 구체적으로 세 가지가 빠져 있다. 첫째는 GPU 메모리 사용량 예측이다. ScaleDL 을 비롯한 다수 연구가 실행 시간만 다루지만, 자원 배정에 실제로 필요한 값은 메모리다. 둘째는 소규모 비균일 GPU 집합에서의 확장성이다. 세대와 모델이 뒤섞인 소수의 장비를 대상으로 하는 예측은 대규모 균일 클러스터를 대상으로 하는 연구와 문제 구조가 다르다. 셋째는 하나의 GPU 위에서 다수 작업이 동시에 실행될 때의 예측이다. 기존 연구는 한 작업이 여러 카드를 쓰는 경우를 다루며, 여러 작업이 한 카드를 나누어 쓸 때의 상호 간섭은 대상으로 삼지 않는다. 세 가지를 함께 다루는 포괄적인 예측에 관한 연구는 조사를 통해 찾지 못했고, 셋 모두 우리 환경에서는 반드시 필요하다.

이를 위해 연구를 진행한다면 「소규모 이기종 GPU 환경에서의 딥러닝 작업 자원 사용량 예측 — 카드 분할 공유를 포함한 GPU 메모리 및 실행 시간 예측 모델」 (영문 Resource Usage Prediction for Deep Learning Workloads on Small-Scale Heterogeneous GPU Clusters: Predicting GPU Memory and Runtime under Fractional Card Sharing)와 같은 새로운 연구가 필요하다. 수행 단계는 네 가지로 나뉜다. 장비 이질성과 분할 비율, 동시 실행 작업 수를 포함하도록 입력 특징을 확장하고, 그 조합에 대한 실측 데이터셋을 구축하며, 메모리 예측의 정확도를 검증하고, 카드 모델 간에 일반화가 가능한지 확인한다. 이는 별도의 개별연구나, 석사 수준의 연구를 통해 다루어야 할 것으로 예상된다.

3.3 특허

첫 번째 사실이 길을 하나 열었다면 두 번째는 다른 길을 막았다. 이전 검토에서는 사용자의 기존 연구환경을 거의 그대로 유지하면서 플랫폼이 GPU 자원을 세밀하게 통제하려면 저수준 GPU 가상화가 사실상 유일하게 남는

방향이라는 결론에 도달해 있었다. 그 무렵 동국대학교에서 열린 PyCon Korea 2026 에 부스 기업으로 참여하였으며, Backend.AI 를 개발하는 회사 Lablup 의 관계자들과 대화를 나눌 수 있었다.

Lablup 은 한국 등록특허 KR102032521B1 과 미국 등록특허 US11221888B2 를 보유하고 있다. 한국 특허는 6 개, 미국 특허는 9 개 청구항으로 구성되며 같은 우선권을 공유하는 해외 출원과 미국 연속출원도 존재한다. 청구된 기술은 컨테이너 내부에서 GPU 관련 라이브러리 호출과 시스템콜을 가로채 자원량과 관련된 인자와 구조체 필드, 반환값을 변경하고, GPU 의 가용량을 관리하여 작업이 요청한 만큼 차감했다가 종료 시 회수한다. 여기서 GPU 자원에는 처리 자원과 GPU memory 가 포함될 수 있고, 단일 GPU 를 여러 컨테이너가 공유하거나 여러 GPU 를 여러 컨테이너에 배정하는 구조도 명세에 담겨 있다. 우리가 도달한 방향과 청구 범위가 강하게 겹칠 가능성이 크다. (별첨 1, 별첨 2) PyCon 현장에서 Lablup 측에 프로젝트의 사정을 설명하고 해당 기술을 사용할 수 있도록 허락을 구했으나 완강한 거절을 받았다. 이 시점에서 저수준 구현은 채택할 수 없는 방향이라고 판단했다.

3.4 HAMi

그런데 조사를 이어가는 과정에서 새로운 사실을 발견하였다. 같은 기능을 수행하는 오픈소스 구현이 이미 존재한다. 최근에 주목받기 시작했으나, 상당한 수준으로 확립되어 있었다. HAMi(Heterogeneous AI Computing Virtualization Middleware)는 Apache 2.0 라이선스를 따르는 프로젝트로, 컨테이너 안에 라이브러리를 주입해 CUDA Runtime 과 CUDA Driver 사이의 호출을 가로채는 방식으로 동작한다. 프로그램이 GPU 메모리 크기를 조회하면 물리적 용량 대신 그 컨테이너에 배정된 용량을 응답하고, 한도를 넘는 할당은 차단한다. 드라이버를 수정하지 않고 커널 계층에서 동작하지 않으며 사용자 코드에도 손대지 않는다. 이 프로젝트는 2026 년 7 월 15 일 CNCF Incubating 프로젝트로 승격되었는데, 이는 실험 단계를 지나 검증 단계에 들어섰음을 뜻한다. 무엇보다, 2026 년 6 월 NVIDIA 가 자사 오픈소스 스케줄러인 KAI Scheduler 본선에 HAMi-core 를 병합했다. NVIDIA 는 직접 구현하는 선택지를 두고 이 프로젝트를 채택했으며, GPU 제조사가 자사 제품에 탑재했다는 점에서 이 방식은 사실상 NVIDIA 의 공인을 받은 셈이다. 기여자는 약 500 명 규모이고 사용하는 조직은 수백 곳이며 NVIDIA 와 AMD 를 포함해 12 종 이상의 가속기를 지원한다.

이 사실은 저수준 구현을 포기하게 만들었던 세 가지 전제를 차례로 무너뜨린다. 첫째 전제는 저수준 분할이 유료 소프트웨어와 특정 GPU 에만 국한된다는 것이었다. MIG 는 데이터센터급 장비를 요구하고 vGPU 는 유료 라이선스를 요구하지만, HAMi 는 둘 중 어느 것도 요구하지 않는다. 공식 자료 기준으로 거의 모든 주류 소비자용 및 데이터센터용 GPU 를 지원하고 MIG 를 요구하지 않으며, 연산 1 퍼센트와 메모리 1MB 단위당 분할을 할 수 있다. 둘째 전제는 소프트웨어의 위험이었다. 16GB 카드에서 두 사용자가 각각 6GB 를 요청했는데 한쪽이 12GB 로 부풀면, 상한이 강제되지 않는 환경에서는 아무 잘못이 없는 다른 사용자의 작업이 실패한다. HAMi 에서는 초과한 쪽이 자기 상한에서 실패하고 옆 사용자는 영향을 받지 않는다. 다른 시스템이 이 문제를 해결해 둔 것도 아니다. Slurm 의 `--gres shard` 기능은 공식 문서상 요청한 VRAM 양이 강제되지 않으며 GPU 에서 사용되는 메모리를 제한할 수 없다고 명시되어 있다. 셋째 전제는 직접 구현의 난이도였는데, 이미 존재하고 오픈소스이며 제조사가 자사 제품에 병합한 구현이 있으므로 만들 이유가 없다. 오픈소스로 공개된 제 3 자 구현을 도입하는 것은 이전에 서술한 특허와 동일 기술을 직접 구현해 배포하는 것과 법적으로 다른 위치에 있으므로, 법적으로 저수준 구현이 충분히 검토 가능한 옵션이 된 것이다.

4. 구현 계획 구체화

3 장에서 확인한 세 가지 사실을 전제로 하면 남은 선택지는 작업 스케줄링과 가상 인스턴스 임대 두 가지로, 이하에서는 각각을 실제로 어떻게 구축할 것인지 구체화한다.

4.1 작업 스케줄링 방식

4.1.1 개요

사용자는 GPU 가 없거나 소형 GPU 만 달린 로그인 장비에 접속해 코드를 작성하고, 실행할 때 명령 앞에 제출 명령어를 붙인다(예. `srun python3 /home/...`). 그러면 플랫폼이 적절한 GPU 장비를 골라 그 장비에서 프로세스를 통째로 실행한다. 홈 디렉터리를 NAS 로 공유하므로 코드는 전송되지 않는다. 이후 결과는 NAS 에 남으며 사용자는 로그인 장비에서 이를 확인한다.

4.1.2 기술 구성

공학적으로는 Slurm 위에 예측 모델을 얹는 구성이 유리하다. 서로 다른 장비에 대한 선언, 연구실 계정 체계 구축, 형평성 계산, 예산 상한 및 선점 정책 설정 기능이 전부 갖추어져 있어 스케줄러를 직접 만들 이유가 없기 때문이다. 3.2 에서 확인했듯 이전 개별연구도, 본 TF 도 이 도구의 존재를 전제하지 않은 채 애플리케이션 계층에 API 를 새로 만드는 방향을 검토하고 있었다.

문제는 예측 부분이다. 예측 기반 배정을 구현하려면 3.2 에서 제시한 연구가 선행되어야 하고, 그 연구는 입력 특징 확장, 실측 데이터셋 구축, 메모리 예측 검증, 카드 모델 간 일반화의 네 단계를 요구한다. 그러나, 예측 모델을 학습하려면 다중 장비와 다중 카드, 분할 공유 환경의 데이터가 필요한데, 그 데이터는 시스템이 운영되거나, 직접 테스트 환경을 구성해 실험을 진행해야 비로소 생긴다. 이 순환은 5 장에서 다시 다룬다.

연구가 끝나면 웹 인터페이스를 구축한다. Open OnDemand 를 포크해 프론트엔드와 백엔드를 분리 구현하는 방안을 검토할 수 있다. 오하이오 슈퍼컴퓨팅 센터가 NSF 지원으로 개발한 Slurm 용 웹 포털, 즉 Slurm 용 프론트엔드로, 콜로라도 볼더, 플로리다 주립, 버팔로, 존스홉킨스, 칼텍, 시카고 등에서 운영 중이어서 참고할 설정 사례가 많다. 브라우저에서 Job 제출 폼과 파일 관리, Jupyter 와 VSCode, 원격 데스크톱 실행을 제공한다. 이 위에 계정 연동, 크레딧 시스템, 연구실 소유권 등과 같은 구체적 구현을 진행한다.

4.2 가상 인스턴스 임대 방식

4.2.1 개요

사용자는 웹에서 티어를 고르고 인스턴스를 생성한 뒤, 고정된 주소로 SSH 로 접속하거나 브라우저에서 바로 작업한다. 인스턴스는 사용자가 종료하기 전까지 유지되며, 그 안에서 무엇을 하든 자유이고 도구도 평소 쓰던 것을 그대로 쓴다. 티어를 바꾸려면 재시작이 필요하지만 저장 공간이 분리되어 있어 작업물은 그대로 남는다. 사용량은 인스턴스의 티어에 따라 크레딧으로 차감하는 방식으로 제한한다.

4.2.2 기술 구성

이 방식은 티어를 어떻게 만드느냐에서 갈린다. 이전에는 GPU 를 카드 단위로만 배정할 수 있다고 판단했으나 HAMi 의 존재로 그러한 제약조건에 더 이상 구애받지 않을 수 있다. 메모리 상한과 연산 비율이 서로 독립된 값이므로, 카드마다 다른 비율을 적용해 같은 티어를 구성할 수 있기 때문이다. 4GB 에 약 10 TFLOPS 인 티어를 만든다고 하면 RTX 3070 에서는 메모리 4GB 에 연산 50 퍼센트를 지정하고 RTX 4090 에서는 메모리 4GB 에 연산 12 퍼센트를 지정한다. 카드마다 TFLOPS 대 GB 비율이 다르기 때문에 단일 분수로는 두 축을 동시에 맞출 수 없고, 축마다 다른 값을 주어야 같은 결과에 도달한다. 다만 SM 비율이 실제 성능을 제대로 대변하는 것은 같은 세대끼리

목을 때이고 세대를 넘으면 어긋나므로, 세대 단위 묶음을 기본으로 삼는다. 들쭉날쭉한 장비로 균일한 티어를 만들어내는 이 작업이 설계의 핵심이며, 장비가 균일한 데이터센터 환경에서는 필요하지 않은 발상이다. 이러한 메커니즘을 통해 보유한 GPU 사양이 제각각인 동국대학교 컴퓨터 AI 학부의 여러 서버를 정규화하여 분배할 수 있게 된다.

티어는 사양에 따라 다르게 다룬다. 저사양 티어는 분할해서 학부 실습처럼 다수가 동시에 소량을 필요로 하는 수요에 대응하고, 고사양 티어는 카드를 통째로 배정해 대역폭 경합과 이웃 간섭 문제를 애초에 만들지 않는다. 이 구분이 시스템 전체의 복잡도를 크게 낮춘다. 공유가 가진 까다로운 성질은 소형 작업에만 국한되고, 장시간 대형 작업은 전용 하드웨어에서 방해 없이 실행되기 때문이다. 저장 공간 설계는 UX 를 중점적으로 고려하여 진행한다. 사용자는 인스턴스 시작에 몇 분이 걸리는 것은 당연하게 여기지만 `pip install` 이 몇 분 걸리면 고장으로 받아들인다. 이에 따라 시간적 비용을 시작과 종료 시점으로 집중하는 방향으로 구현한다. 인스턴스가 살아 있는 동안 홈 디렉터리와 작업 파일은 노드 로컬 디스크에 두고, 시작할 때 NAS 에서 가져오고 정지하거나 티어를 변경할 때 NAS 로 되돌린다. 이때 파일 하나하나를 옮기면 네트워크 파일시스템의 메타데이터 왕복 비용이 그대로 발생하므로 단일 이미지 파일이나 아카이브 단위로 옮긴다. 데이터셋은 크기가 크고 순차 읽기가 대부분이라 네트워크 파일시스템이 잘 처리하는 형태이므로 NAS 에 읽기 전용으로 두고 복사하지 않는다. 이 구조에서는 노드 로컬 디스크 용량이 새로운 설계 변수가 된다. 인스턴스 하나가 자기 작업 공간을 해당 장비의 디스크에 유지하므로 장비당 수용 가능한 인스턴스 수가 GPU 용량과 별개로 디스크 용량에도 걸리며, 티어별 디스크 할당량을 정할 때 이를 반영한다.

네트워크는 하나의 고정 주소로 통일한다(예. `compute.dongguk.edu`). SSH 는 URL 경로로 분기할 수 없으므로 사용자명 기반 라우터를 두어 학번에 해당하는 인스턴스로 연결하며, 인스턴스가 어느 장비에 있든 접속 주소는 변하지 않는다. TensorBoard 나 Jupyter 같은 웹 도구를 사용하기 위해서 네트워크 기능이 필요하므로, AWS 의 VCP 와 유사한 방식으로 인스턴스별 리버스 프록시를 대시보드에서 설정할 수 있도록 한다. 모든 것이 하나의 컨테이너 안에서 실행되므로 라우팅 계층이 하나뿐이므로, 설정 기능만 제공해도 네트워크를 요구하는 대부분의 ML 도구가 사용 가능하다. 단, 프로덕션 서비스용이 아니므로 VCP 에 비해 대단히 단순화된 구성으로 충분하다.

5. 두 구현계획의 비교

2 장이 방식 자체의 일반적 성질을 다루었다면, 이 장은 4 장에서 확정한 계획을 실제로 구축했을 때의 의미와 장단점을 검토한다.

5.1 작업 스케줄링

이 방식은 크게 세 가지의 이점을 가져다 준다. 첫째, 작업 단위로 배정하므로 idle 자원이 최소화되어 자원 활용률이 가장 높다. 둘째, 작업을 제출받아 종료 시간을 예측해 스케줄링한다는 점에서 지도교수의 초기 구상에 가장 부합한하며, 예측이 배정 결정을 직접 구동하므로 이전 개별연구를 가장 크게 활용한다. 셋째, 다양한 장비에 대한 대응이 수많은 대학의 운영으로 이미 검증되어 신뢰성이 높다.

그러나, 사용자 입장에서 문제가 존재한다. 현대 AI 작업 도구는 프로세스가 사용자가 앉아 있는 장비에 있다고 전제하는데, 작업을 제출하는 방식은 그 전제를 위반한다. 이 방식은 필연적으로, 실제로 GPU 를 사용하는 코드가 실행되는 장비가 숨겨진 네트워크 뒤에 놓이기 때문이다. 구체적인 사례를 살펴보면, 우선 TensorBoard 는 사용이 완전히 불가능하며, 로그만이 NAS 에 남아 이를 확인하는 것으로 대체해야 한다. Jupyter 는 셸 사이에 GPU 상태를 유지하는 장기 프로세스여서 사용이 불가능하다. VSCode 원격 개발은 장비에 접속하는 방식이라 다른 장비에서 도는

GPU 프로세스를 보지 못한다. pdb, 프로파일러, 실행 중 프로세스 접속이 전부 불가능하다. `nvidia-smi`와 `nvidia-top`의 사용 역시 불가능하다. gradio 같은 데모 포트는 실행 장비에 열리므로 별도 전달이 필요하다. 사용자는 자기 코드의 호출 지점은 고칠 수 있지만 자신이 관리하지 않는 두 장비 사이의 네트워크 경로는 고칠 수 없으므로, 이에 대한 해결은 오로지 본 솔루션을 개발 및 유지보수하는 주체에게 있다. 그러나 이러한 네트워크 문제의 특성상 도구 하나당 문제 하나를 풀어야 하므로, 이를 해결하기 위해서는 대단히 많은 경우의 수를 고려하고 충분한 유지보수 노력이 필요하다.

이는 본 방식이 가장 잘 처리할 수 있는 장시간 작업에서 타격이 크다. 사흘짜리 학습을 실행하는 동안 로그인 장비에서는 GPU가 실제로 포화 상태인지 데이터 로더에 막혀 있는지, 메모리가 OOM 방향으로 증가하고 있는지, 작업이 멈춘 것인지 그냥 느린 것인지, Loss의 변화는 어떠한지, 발열로 성능이 떨어지고 있는지를 확인할 방법이 없다. 이를 도구 없이 로그만으로 직접 판단해야 한다. Loss의 추이를 그래프로 보려면 로그 레벨을 Verbose로 올린 후 실시간 파서를 직접 구현해야 한다. 이 부분에 대해 교내 석사 과정 대학원생 몇 명을 대상으로 진행한 간단한 유선 면담에서는 연구자 입장에서 수용하기 어렵다는 반응이 나왔다.

개발 측면에서 또다른 문제가 있다. 예측 기반 배정을 구현하려면 3.2에서 제시한 연구가 완료되어야 하는데, 그 연구에 필요한 다중 장비와 다중 카드와 분할 공유 데이터는 시스템이 운영되거나, 새로운 실험을 설계해야 얻을 수 있다. 이에선 충분한 시간과 노력이 별도로 필요하다.

행정 부담도 줄어들지 않는다. Slurm은 정책을 표현할 수단만 제공하기 때문이다. 연구실별 지분, 반감기, 다섯 개의 우선순위에 대한 가중치, QoS 등급과 부여 대상, 파티션 우선순위, 선점 모드, GPU-사용시간에 대한 예산을 전부 우리가 정해야 한다. 이는 기술적인 문제가 아닌 행정적인 문제이므로, 결국 시스템을 최종 도입하기 전에 정책에 대한 충분한 설문과 합의가 필요하다.

그러나 이러한 비용을 치러야 함에도 본 방식은 충분한 이점을 가진다. 순정 Slurm에 Open OnDemand를 얹으면 대기열, 이질적 장비 대응, 형평성, 예산, 웹 인터페이스가 이미 상당 부분 완성된다. 우리가 추가해야 하는 것은 예측 기반 배정, 연구실 소유권 및 기여도 반영, 학내 계정 연동, 크레딧 회계 네 가지인데, 연구가 필요한 첫 번째를 제외한 나머지는 설정 작업과 인터페이스 커스터마이징에 해당한다.

현재 Princeton, Northwestern, Utah 등의 대학이 이미 이 방식으로 운영하며, 특히 유타대학교 CHPC와 메릴랜드대학교 UMIACS는 장비를 두 개의 대기열에 동시 등록하고 소유 연구실의 작업이 들어오면 외부 작업을 큐로 되돌리는 방식으로 소유권을 유지한다. 국내 대학에서 이 방식을 주 운영 방식으로 채택한 사례는 조사 범위에서 찾지 못했다.

5.2 가상 인스턴스 임대

이 방식은 수요 범위가 넓다. 커널이나 드라이버 자체를 연구하는 경우가 아니면 사실상 모든 수요를 커버할 수 있다. AI와 컴퓨터 비전 연구부터 학부 실습까지 포함되며, 커널 계층 접근은 베어메탈 환경이 아닌 이상 어느 방식에서도 불가능하므로 실질적 제약이 되지 않는다. 그중에서도 교육 수요와의 적합성이 특히 높다. 동국대학교의 IT 관련 학과들이 컴퓨터 AI 학부로 통폐합되면서 여러 전공과목에 대한 실습이 학부생 실습 서버를 쓰고 있는데, 본 방식을 통해 이를 GPU 실습에 대해 그대로 확장할 수 있다. 예컨대, 머신러닝 수업에서 40명이 동시에 실습 코드를 실행한다고 가정하자. 스케줄링 방식에서는 대기열과 우선순위 문제가 되지만 임대 방식에서는 전원에게 동일한 소형 티어를 배정하는 것으로 끝난다. 국민대학교 사례가 이를 뒷받침한다. GPU 서버 3대로 80명 이상이 동시에 모델링 실습을 수행했고 전담 관리 인력 없이 운영되었다는 기록이 있다. 또한, TensorBoard, nvidia-top 등의 도구 사용이 가능하다는 장점이 있으며, 그리고 무엇보다 친숙도에서도 유리하다. 하나의 컨테이너, 하나의 장비, 하나의 라우팅 계층으로

구성되기 때문에 거의 모든 머신러닝 보조 도구가 그대로 동작한다. 그리고 학부생과 대학원생 대부분이 AWS 나 GCP 나 OCI 가운데 하나는 써 본 경험이 있고 인스턴스 구축과 VCP 등 네트워크 설정을 다루어 보았으므로 별도의 학습이 거의 필요하지 않다.

행정적 고민에 대한 부담은 스케줄링보다 크게 가볍다. 첫째, 크레딧이라는 단일 단위를 사용하면 되므로 조정해야 할 값의 차원이 낮아 Slurm 처럼 여러 값의 상호작용을 생각할 필요성이 없다. 둘째, 효과가 직관적이어서 티어 단가를 올리면 사용이 줄어들고 별도의 시뮬레이션이 필요하지 않다. 셋째, 파장이 국소적이어서 한 티어의 단가를 바꾸어도 다른 티어 사용자는 영향을 받지 않는다. 넷째, 수정이 비파괴적이어서 단가 혹은 크레딧 지급량을 바꾸어도 실행 중인 작업에는 아무 일도 일어나지 않고 다음 배정부터 적용된다. 무엇보다 선점 정책이 필요하지 않아서, 행정적으로 가장 다루기 어려운 문제를 해결할 필요성이 적다. 크레딧의 배분과 조정은 교수진 회의를 통해 이루어지며, 특정 연구실에 특정 기간 동안 추가로 지급하거나 배분량을 인상하고 삭감하는 등의 결정이 교수 상호 간의 합의로 이루어질 수 있다.

본 방식을 구현하게 된다면, Slurm 과 같은 완성된 솔루션 대신 HAMi 와 같은 저수준 오픈소스 프로젝트를 기반으로 직접 구현해야 한다. Backend.AI 와 같은 기존 오픈소스 솔루션을 도입하면 세션 모델, 웹 인터페이스, 이미지 카탈로그, 컨테이너 수명 관리, 사용자 및 그룹 관리, 저장 공간이 이미 갖추어진다. 그런데 정작 GPU 분할 기능인 fGPU 가 오픈소스판에 포함되지 않으며, 이 부분이 해당 회사의 특허 청구항이기도 하다. 직접 구현해야 한다는 것은 구현 노력을 그만큼 소모해야 한다는 뜻이기도 하지만, 직접 구현함으로써 네 가지 이점을 얻을 수 있다. 첫째, HAMi 로 fGPU 에 해당하는 기능을 확보해 유료 기능을 대체할 수 있으며, 이는 성균관대학교가 GPU 당 시간당 2,200 원으로 운영하는 바로 그 기능을 무료로 도입할 수 있게 된다는 의의가 있다. 둘째, 세대와 모델이 뒤섞인 기증 장비를 하나의 티어 체계로 정규화하는 균일성을 확보한다. 기성 제품은 장비가 균일한 환경을 전제하므로 이 기능을 제공하는 제품이 없으며, 우리 상황에서만 의미를 갖는 이 부분이 직접 구현을 채택할 이유가 된다. 셋째, 기존 클라우드 서비스와 달리 사용자가 필요에 따라 티어를 올리고 내릴 수 있다. 넷째, 크레딧 배분 규칙과 연구실별 기여도 반영, 학내 계정 연동처럼 제품 설정으로 표현할 수 없는 우리 학교의 규칙을 직접 반영할 수 있다. 상용 제품 구매와 비교하더라도, 성균관대학교 규모의 과금 운영을 전제하지 않는 이상 유료 기능 하나를 위해 상용 계약을 맺기보다 오픈소스 조합으로 같은 기능을 확보하는 편이 1.1 의 제약 조건에 부합한다. 4.2.2 에서 제시한 바와 같이, 서로 다른 그래픽카드로 유사한 성능을 내는 인스턴스를 분할해 내려면, 선제적으로 벤치마킹이 필요하다. 이전 개별연구의 메트릭 수집기는 이미 실제 장비에서 실행 시간과 SM 활용률을 수집하고 있으므로, 이 정보는 해당 연구를 활용하여 측정할 수 있다.

이 방식의 단점 역시 다양하게 존재한다. 자원 활용률이 스케줄링보다 낮아서 인스턴스를 켜두고 쓰지 않아도 자원이 묶인다. 다만 이 문제는 크레딧 구조가 보완할 수 있다. 켜둔 시간만큼 크레딧이 계속 차감되므로 사용자에게 미사용 시 종료할 유인이 생기기 때문이다. 그 밖에 티어 변경 등 인스턴스가 물리적 기기 사이를 이동해야 할 때 재시작이 필요하며, 이는 파일이 네트워크로 전송되어야 하므로 시간이 상당히 오래 걸릴 수 있다. 또한, 메모리 대역폭까지 나누지는 못해 같은 카드의 여러 사용자들이 대역폭을 공유하게 된다는 문제가 있다. 고사양 티어를 분할하지 않는 정책으로 완화할 수 있으나, 실습 및 간단한 작업에 사용하게 되는 저티어 인스턴스에서 문제가 될 수 있다.

이 방식을 채택한 사례는 주로 국내에 집중되어 있다. 성균관대학교가 Backend.AI 로 유료 과금 운영을, 국민대학교가 Backend.AI 로 서버 3 대에서 80 명 동시 실습을, KAIST AI 대학원이 VESSL AI 로 GPU 800 장 이상, 연구팀 20 개, 대학원생 300 명 이상 규모를 운영하며 월간 GPU 사용 시간과 동시 작업 수 할당량을 적용하고 있다. 고려대학교는 공동기기원, POSTECH 역시 이 방법을 사용해 무상 개방하는 형태를 채택했다.

6. 결론

조사 내용을 종합하여 판단한 결과, 현재의 인력과 기간 안에서 실현 가능하고, 학부 실습부터 연구까지 수요를 포괄하며, 연구자가 쓰던 도구를 그대로 쓸 수 있다는 점에서, 가상 인스턴스 임대 방식이 보다 적합한 것으로 보인다.

이 선택은 이전 개별연구의 향후 방향을 확장한다. 임대 방식으로 시스템을 먼저 구축하고 이후 백엔드 러너에 성능 지표 수집 기능을 추가하는 패치를 적용하면 실사용 데이터를 쌓을 수 있기 때문이다. 여러 장비와 여러 카드 모델, 하나의 카드 안에서 동시 실행되는 다중 프로세스에 걸친 실제 워크로드 기록 등을 축적할 수 있으며, 이를 통해 인위적으로 생성한 데이터셋으로는 확보하기 어려운 다양성을 확보할 수 있다. 기존 연구가 데이터센터급의 균일한 환경을 전제하는 상황에서 이런 환경의 실측 자료는 공개된 것이 없다. 이 데이터를 바탕으로 3.2의 연구를 수행하면 독립된 연구 성과에 이를 수 있을 것으로 보인다.

위와 같은 사항들을 고려하여, 담당교수의 최종 결정에 참고가 되기를 바란다.

별첨

별첨 1. 대한민국 등록특허공보 KR102032521B1, 「컨테이너 기반의 GPU 가상화 방법 및 시스템」

([별첨 1] KR102032521B1 컨테이너 기반의 GPU 가상화 방법 및 시스템.pdf)

별첨 2. 미국 등록특허공보 US11221888B2, 「Method and system for GPU virtualization based on container」

([별첨 2] US11221888B2 Method and system for GPU virtualization based on container.pdf)

별첨 3. 성균관대학교 슈퍼컴퓨팅센터, 「GPU 클러스터 사용 안내」, 2022. 9. 1. 기준

([별첨 3] 성균관대학교 슈퍼컴퓨팅센터 GPU 클러스터 사용 안내(2022.9.1).pdf)

참고자료

특허 및 제품

[1] 대한민국 등록특허 KR102032521B1.

<https://patents.google.com/patent/KR102032521B1/ko>

[2] 미국 등록특허 US11221888B2.

<https://patents.google.com/patent/US11221888B2/en>

[3] Lablup, Backend.AI GPU 가상화(fGPU) 소개.

<https://www.backend.ai/platform/gpu-virtualization>

[4] Lablup, Backend.AI CUDA Accelerator Plugin (오픈소스판).

<https://github.com/lablup/backend.ai-accelerator-cuda>

[5] Lablup, Backend.AI 라이선스 고지.

https://github.com/lablup/backend.ai-docs-webui/blob/main/docs/license_agreement/license_agreement.rst

[6] Lablup, Backend.AI 행사 참여 목록.

<https://www.backend.ai/ko/event>

[7] 동국대학교 컴퓨터공학전공, PyCon Korea 2026 개최 안내.

<https://cs.dongguk.edu/article/noti/detail/1408>

저수준 GPU 분할

[8] Project-HAMi 공식 사이트.

<https://project-hami.io/>

[9] Project-HAMi, GPU 가상화 동작 원리.

<https://project-hami.io/docs/core-concepts/gpu-virtualization>

[10] CNCF, HAMi becomes a CNCF incubating project (2026. 7. 15.).

<https://www.cncf.io/blog/2026/07/15/hami-becomes-a-cncf-incubating-project/>

[11] Project-HAMi, HAMi-core Adopted by NVIDIA KAI Scheduler (2026. 6.).

<https://project-hami.io/blog/hami-core-adopted-by-nvidia-kai-scheduler>

[12] NVIDIA, Open Sourcing the Run:ai Scheduler (KAI Scheduler).

<https://developer.nvidia.com/blog/nvidia-open-sources-runai-scheduler-to-foster-community-collaboration/>

작업 스케줄링

[13] SchedMD, Slurm Generic Resource (GRES) Scheduling.

<https://slurm.schedmd.com/gres.html>

[14] SchedMD, Slurm Preemption.

<https://slurm.schedmd.com/preempt.html>

[15] University of Utah CHPC, Slurm job preemption.

<https://www.chpc.utah.edu/documentation/software/slurm-job-preemption.php>

[16] University of Maryland UMIACS, SLURM Priority.

<https://wiki.umiacs.umd.edu/umiacs/index.php/SLURM/Priority>

[17] Open OnDemand (University of Colorado Boulder 운영 문서).

https://curc.readthedocs.io/en/latest/open_ondemand/index.html

국내 대학 운영 사례

[18] Lablup, 국민대학교 도입 사례.

<https://www.backend.ai/resources/kmu-casestudy>

[19] VESSL AI, KAIST AI orchestrates 800+ GPUs.

<https://blog.vessl.ai/en/posts/kaist-ai>

[20] VESSL AI, Quotas and limits.

<https://docs.vessl.ai/guides/clusters/quotas>

[21] 고려대학교, GPU 클라우드 서비스 출범.

<https://www.korea.ac.kr/bbs/ko/42/120489/artclView.do>

[22] 성균관대학교 슈퍼컴퓨팅센터.

<https://supercom.skku.edu>

원격 GPU 가상화 및 성능 예측 연구

[23] rCUDA, A complete and efficient CUDA-sharing solution for HPC clusters.

<https://www.sciencedirect.com/science/article/abs/pii/S0167819114001227>

[24] Chen, X. et al., PerfSeer: An Efficient and Accurate Deep Learning Models Performance Predictor, IJCAI 2025.

<https://arxiv.org/abs/2502.01206>

[25] ScaleDL: Towards Scalable and Efficient Runtime Prediction for Distributed Deep Learning Workloads.

<https://arxiv.org/abs/2511.04162>

[26] HETHUB: A Distributed Training System with Heterogeneous Cluster for Large-Scale Models.

<https://arxiv.org/abs/2405.16256>

[27] Cephalo: Harnessing Heterogeneous GPU Clusters for Training Transformer Models.

<https://arxiv.org/abs/2411.01075>

[28] HARP: Orchestrating Automated Parallel Training on Heterogeneous GPU Clusters.

<https://arxiv.org/abs/2509.24859>